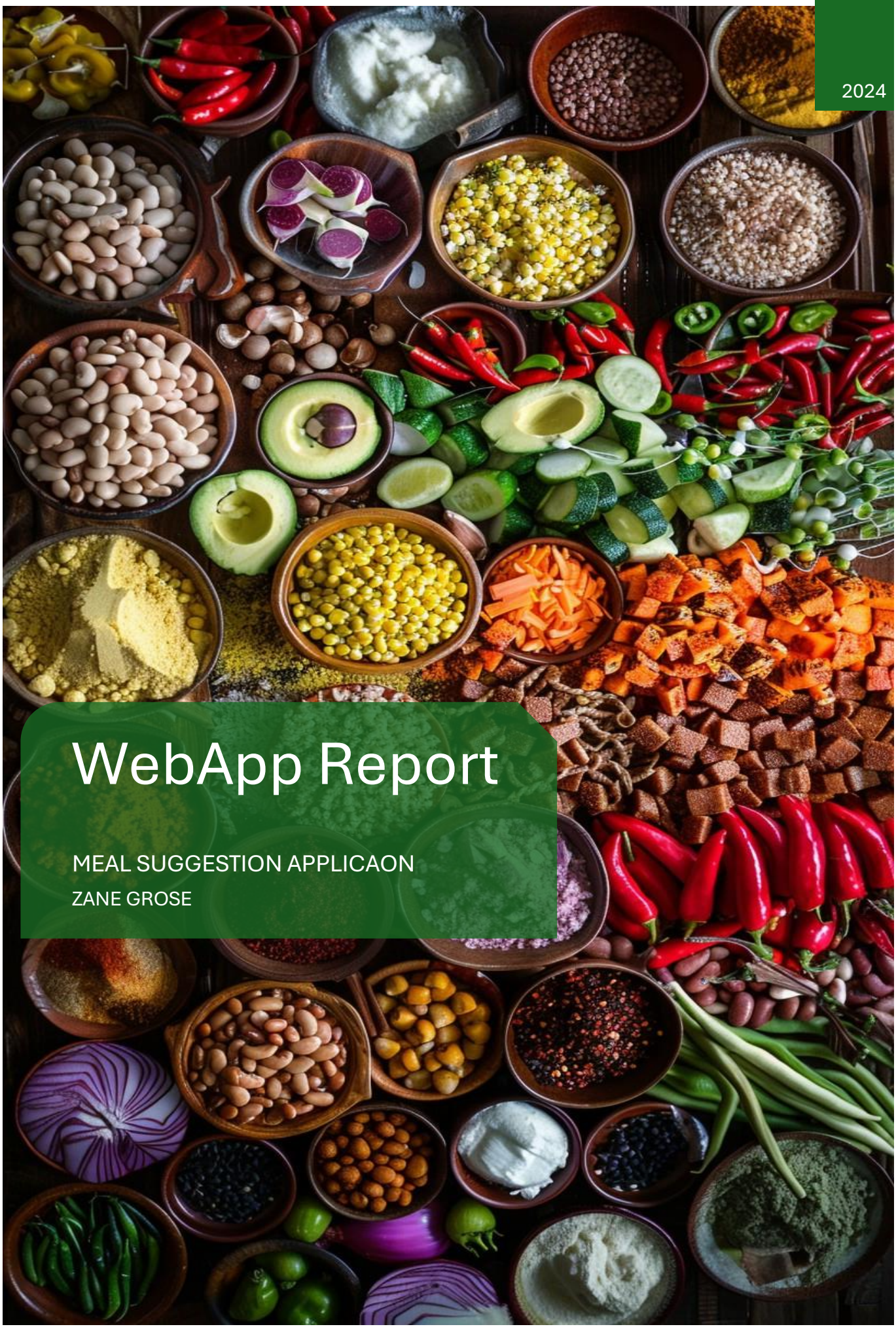
2024



WebApp Report

MEAL SUGGESTION APPLICAON

ZANE GROSE

Contents

Background of Application: 1
Technical Features: 1
Pseudocode: 1
Diagrams: 3
  Wireframes: 3
  Component & Sequence Diagrams: 4
Reflection: 6
Website Link: 6

Background of Application:

Imagine opening your fridge or pantry and seeing a mix of different ingredients, creating a confusion around what to cook. This dilemma was the direct problem that was behind the conception of this application. The proposed application will users decide what to cook based on their selected ingredients. Users can select from a list of ingredients; the application will then determine which meal has the most matching ingredients based on the users selected ingredients. This app leverages a great website that manages a database with information on 303 meals and over 600 ingredients.

Technical Features:

This application is a front-stack application, only utilizing TheMealDB external database. The application 'App' component will provide the user with two lists of ingredients when initially loading the website. 'Main' ingredients are displayed from a custom .txt file (MainIngredients.txt), where ingredients were manually selected based on their return values from TheMealDB API. Whereas 'Other' ingredients are fetched from TheMealDB API, with duplicate records in the .txt file were removed from the 'Other' ingredients list. Main ingredient selection is restricted to 3 ingredients and a user may only get a suggested meal once every 30 seconds, this is due to the calling limitations of the API.

Pseudocode:

- User pressed button
- FetchingMeal = true
- Initialize maxCount
- Initialize mealCounts
- Initialize suggestedMeal
- For each Selected Main Ingredients
    - Call 'MealByMainIngredient' from API
        - For each returned meal
            - Initialize counter = 0
            - Fetch returned mealIngredients
            - For each ingredient of mealIngredients
                - If selectedIngredient == mealIngredient
                    - Count++
                - If count > maxCount
                    - maxCount = count
                    - mealCounts = suggestedMeal
                - else if count == maxCount
                    - add mealDetails to mealCounts
            - if mealCount.length > 1
                - get random meal in mealCounts

- setSuggestedMeal(suggestedMeal)
- fetchingMeal = false

Diagrams:

Wireframes:

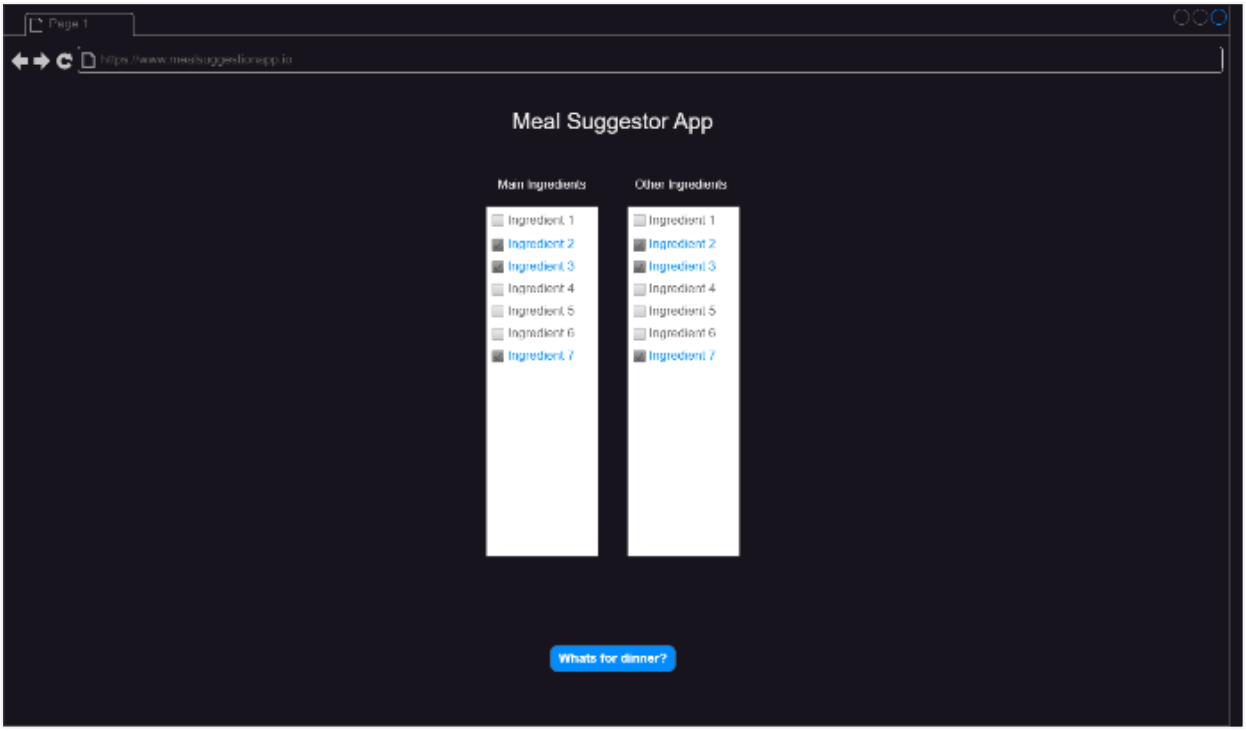


Figure 1: App ingredients list wireframe

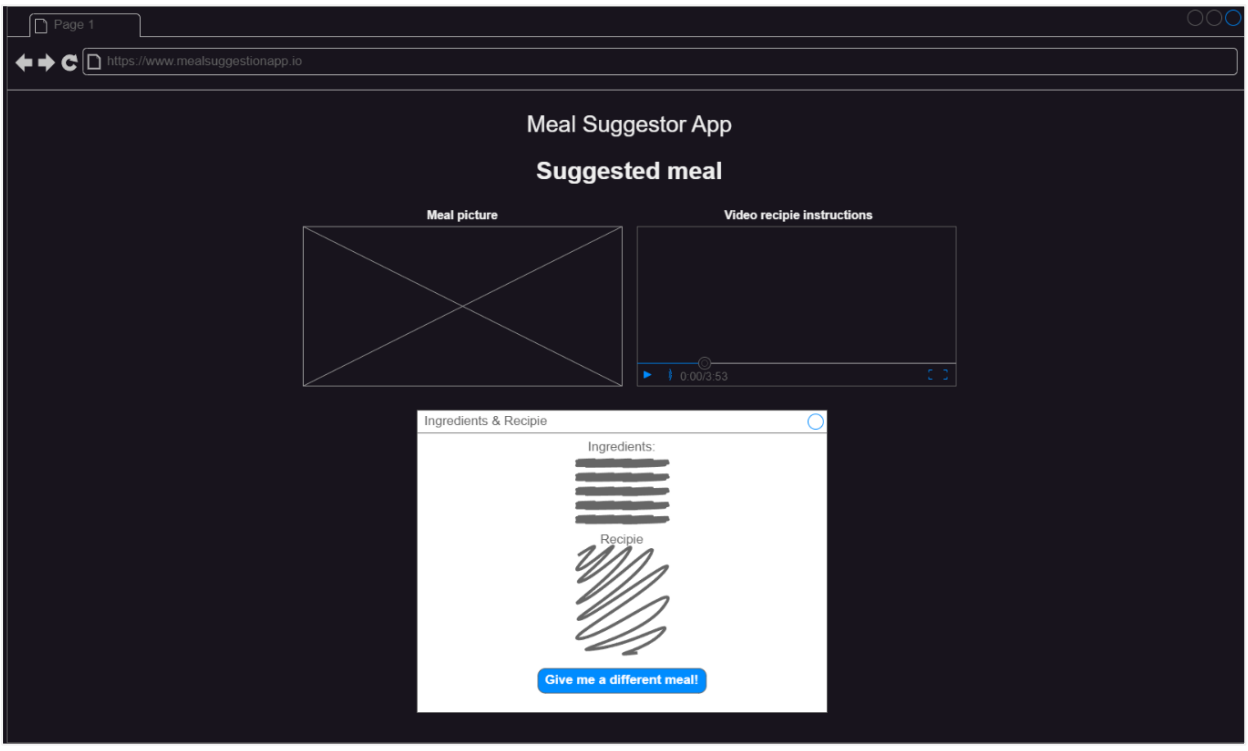


Figure 2: Suggested meal display wireframe

These wireframes were created as a general layout of the application, which assisted me in visualizing the organization of the information on the page.

Component & Sequence Diagrams:

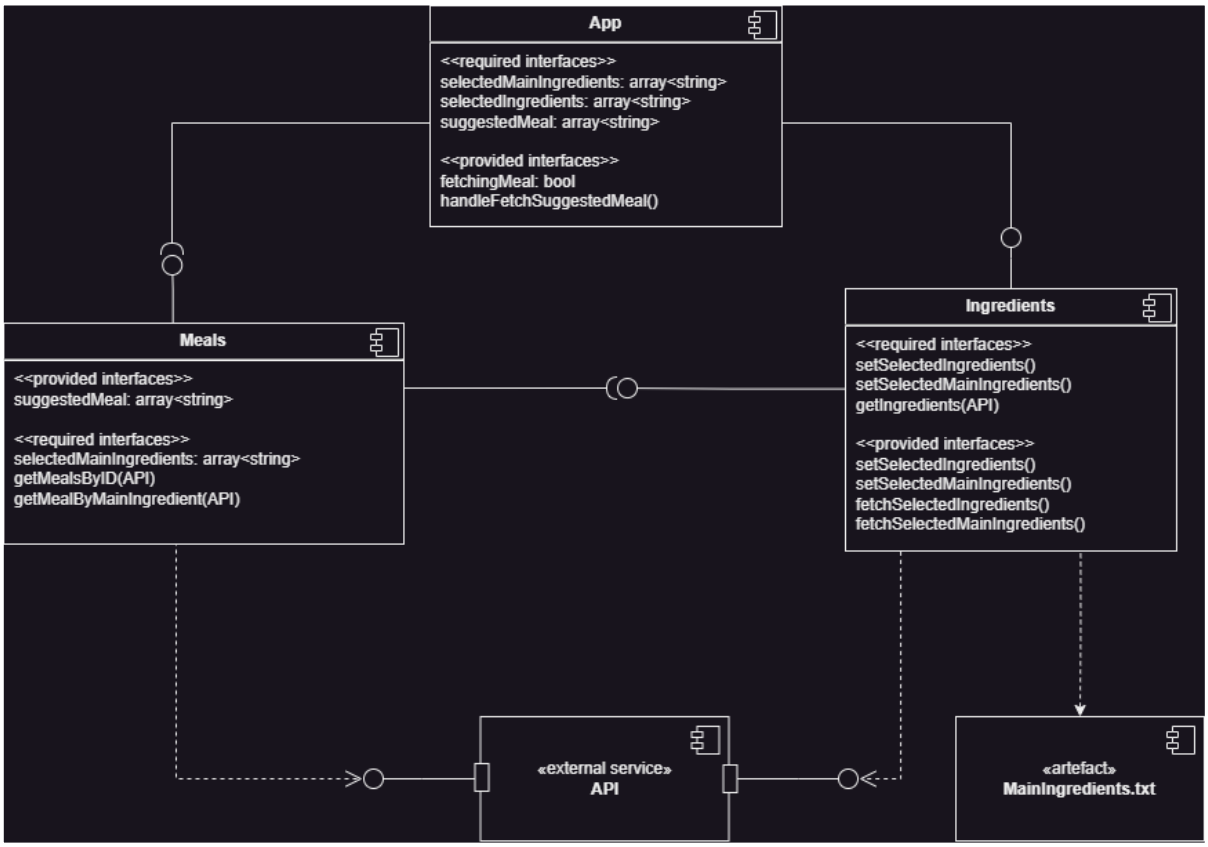


Figure 1: Component diagram of meal-suggestion application

Figure 3 shows the interactions between components in the application.

App passes down its 'main' and 'other' ingredient states to the ingredients component, these two components rely on each other. The ingredients component will handle the changes in the checkboxes for each ingredient list. The App also requires an interface from the Meal component, this is the 'suggestedMeal' variable, which is internally set within the Meal components logic with fetchSuggestedMeal.

The components Meals & Ingredients both require information from the API external service, this is denoted via the dotted line going from the components to the external service. Ingredients also obtains data from the MainIngredients.txt file.

Furthermore, the Meals component requires to know what main ingredients have been selected, so then the logic can call the API based on the selections.

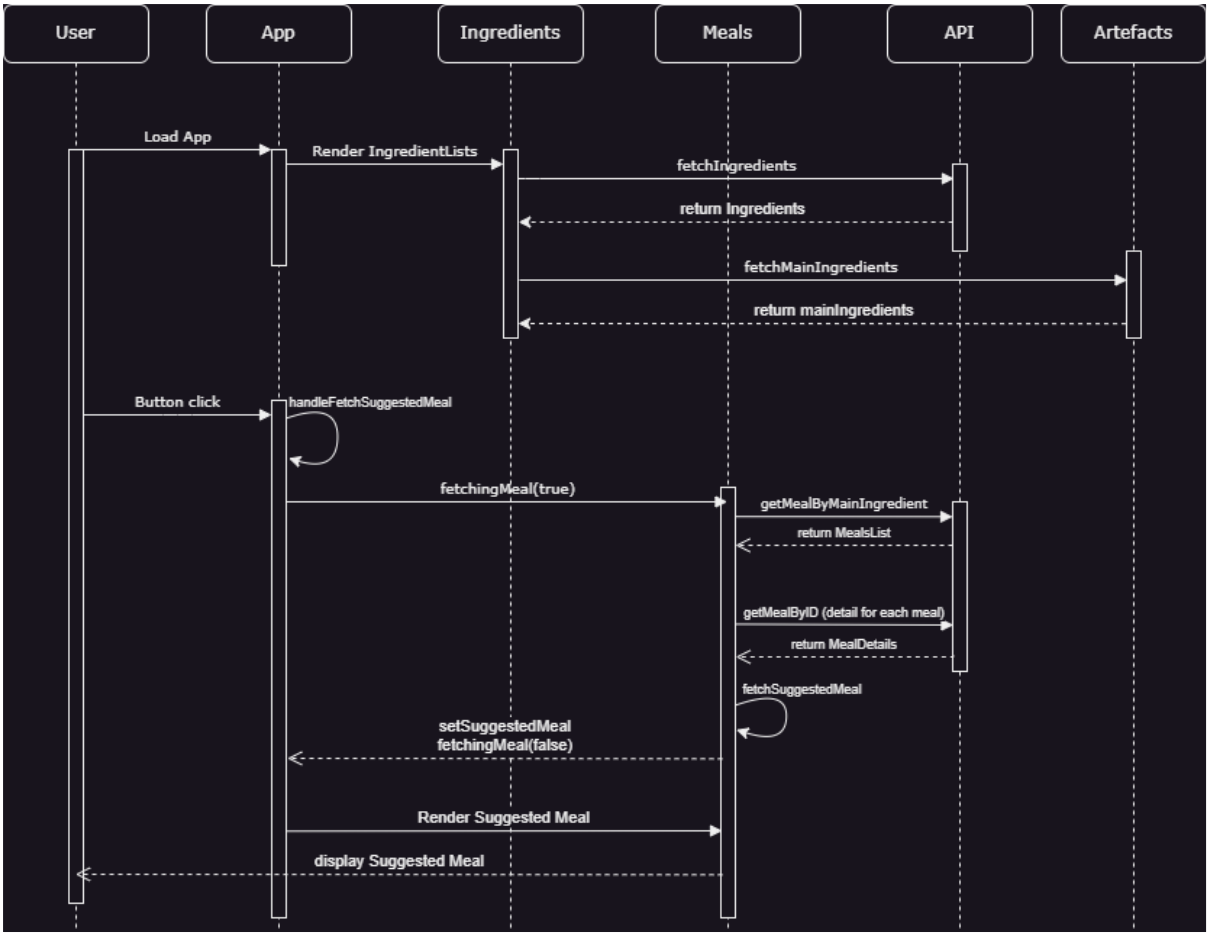


Figure 2: Sequence diagram of processes within each component

In *Figure 4*, we can see how the application components interact with each other. A user will load the application, this triggers the ingredients component to fetch the ingredients lists, which is received from the API and Artefacts(.txt file) in my project. When the user presses the button to get a suggested meal, the App component will handle the flag for selecting a meal, which changes the fetchingMeal state to 'true'. This flag is passed down to the Meal component, which the queries the API in the 'fetchSuggestedMeal' function. After this process is finished, the Meal component will set the suggested meal and change the fetchingMeal state back to 'false'. When the App has the suggested meal, it asks the Meal component to render the suggested meal, then in turn the Meal component displays the suggested meal to the user.

Reflection:

After completing the project, there were some issues that could be addressed. Firstly, the other ingredients list is quite big and can get very annoying to look at. To solve this, I would section all the ingredients off into their specific categories (fruit, vegetable, spice, baking, etc..). This would make it much easier for the user to select ingredients and not have to scroll through the list, diligently looking for their ingredients. The logic behind the application works well, but this can be iterated upon. There is an API call from TheMealDB that allows to search for a meal based on multiple ingredients, this could be incorporated into the application to lower API calls and give more precise meal suggestions based on the selected ingredients, though could be hard to keep the randomness of the logic. Another consideration is the number of calls per minute using my developer key. As the limit is 100 calls per minute, if I have just 5 people getting meals from my website at the same time, the call amount can very quickly go above 100. This issue seems manageable currently, but for professional publication, the current setup may not be the best. The UI design of the website is quite simple, the colours contrast can be bright for some people (it was for me). In further iterations, this would be expanded upon to create more consistent contrast between UI elements and background colours/images. Additionally changes to the UX flow of the application could provide benefits.

Website Link:

To access the website, please visit https://meal-suggestor.web.app/. If the application is down, or isn't working, please contact me at zane.grose@hotmail.com.